Complete finite programs for incidence geometries

Human Compression project

11 October 2026

Program roles

Four complete programs provide mathematical inputs: the exceptional linear and small-unitary actions, the deleted-pencil residue at order four, the native Witt designs and stabilizers, and the Mathieu chamber fibers. The independent witness validator and the generator of the literal Lean tables complete the computational bridge for the latter models. Seven further programs test consequences and source corrections proved in the written papers. They are retained in full and distinguished from the required mathematical inputs. The operational driver `check.py` verifies the input hashes, runs these files, rebuilds the formal certificates and rejects specified altered inputs. It is included separately, rather than counted as another mathematical lemma.

The finite programs do not replace the proofs for every prime power. Classical Mathieu actions, the GAP permutation-group algorithms and the DESIGN/GRAPE full automorphism calculation remain explicit computational inputs. The exact roles and filenames are bound in the inventory.

1. phase265_exceptional.g

Role: required mathematical computation. L009 exceptional A7 natural action; explicit UH(2) to AG(2,3) design map; whole file also includes source controls.

```
# Complete exceptional group/design interfaces and literal source controls.
Need:=function(b,s) if not b then Error(s);fi;end;
F:=GF(2);vec:=Filtered(Tuples(Elements(F),4),v->not IsZero(v));
G:=Action(GL(4,2),vec,OnRight);A8:=AlternatingGroup(8);
iso:=IsomorphismGroups(G,A8);Need(iso<>fail,"GL4,2 to A8 isomorphism");
H:=PreImage(iso,Stabilizer(A8,8));
Need(Size(G)=20160 and Size(H)=2520,"exceptional group orders");
T:=Filtered(Tuples([1..15],3),t->Length(Set(t))=3 and vec[t[1]]+vec[t[2]]<>vec[t[3]]);
Need(Length(T)=2520 and Set(Orbit(H,T[1],OnTuples))=Set(T),"all noncollinear triples");
Need(Length(Orbit(H,[1,2],OnTuples))=210,"all distinct nonzero vector pairs");
Print("PHASE265_A7_PASS PG3,2_noncollinear2520 affine_binary_pair210\n");
# U_H(2) as the unique displayed nine-point affine-plane design.
q:=2;K:=GF(4);els:=Elements(K);z:=Zero(K);o:=One(K);pv:=[];
for x in els do for y in els do Add(pv,[o,x,y]);od;od;
for x in els do Add(pv,[z,o,x]);od;Add(pv,[z,z,o]);
pts:=Filtered(pv,v->Sum(v,x->x^(q+1))=z);blocks:=[];
for c in pv do
 B:=Filtered([1..Length(pts)],i->Sum([1..3],j->pts[i][j]*c[j])=z);
 if Length(B)=3 then AddSet(blocks,B);fi;
od;
Need(Length(pts)=9 and Length(blocks)=12,"small unital sizes");
V:=Tuples(Elements(GF(3)),2);AGlines:=Set(Concatenation(List(V,v->List(Filtered(V,w->not
IsZero(w)),w->Set(List(Elements(GF(3)),a->Position(V,v+a*w)))))));
Need(Length(AGlines)=12,"affine three lines");
Tsrc:=First(Combinations([1..9],3),t->not t in blocks);
Tdst:=First(Combinations([1..9],3),t->not t in AGlines);
restsrc:=Difference([1..9],Tsrc);restdst:=Difference([1..9],Tdst);found:=fail;
f:=[];
for per in PermutationsList(restdst) do
 f:=List([1..9],x->0);
 for i in [1..3] do f[Tsrc[i]]:=Tdst[i];od;
```

```
 for i in [1..6] do f[restsrc[i]]:=per[i];od;
 if Set(List(blocks,B->Set(List(B,x->f[x]))))=AGlines then found:=f;break;fi;
od;
Need(found<>fail,"literal unital-affine incidence isomorphism");
Print("PHASE265_UH2_PASS explicit_point_bijection",found," all12blocks\n");
# Field automorphisms missing from a literal AGL label.
K:=GF(4);V:=Tuples(Elements(K),2);z:=Z(4);zero:=Zero(K);one:=One(K);
lines:=Set(Concatenation(List(V,v->List(Filtered(V,w->not
IsZero(w)),w->Set(List(Elements(K),a->Position(V,v+a*w))))));
frob:=PermList(List(V,v->Position(V,List(v,x->x^2))));
Need(ForAll(lines,L->OnSets(L,frob) in lines),"Frobenius line preservation");
fixed:=List([[zero,zero],[one,zero],[zero,one]],v->Position(V,v));
Need(ForAll(fixed,i->i^frob=i) and Position(V,[z,zero])^frob<>Position(V,[z,zero]),"not K-affine linear");
# A forward incidence morphism need not respect the negative incidence clause.
# The triangle K3 has point0 on line{0,1}; collapsing each input type there
# preserves all input incidences but destroys the displayed triangle edge.
Need(0 in [0,1] and not 2 in [0,1] and 0 in [0,2],"source triangle edge");
Need(not (0 in [0,1] and not 0 in [0,1]),"constant morphism destroys edge");
Print("PHASE265_CONTROLS_PASS Frobenius_not_AGL literal_functor_counterexample\n");
QUIT;
```

2. phase265_residues.g

Role: required mathematical computation. L009 UH(4) deleted-pencil diagram and exclusion of a part-swapping automorphism.

```
# Exact deleted-pencil residue of the classical Hermitian unital U_H(4).
# No original source table is used as input.
need:=function(b,s) if not b then Error(s);fi;end;
q:=4;K:=GF(q^2);els:=Elements(K);z:=Zero(K);o:=One(K);
pv:=[];
for x in els do for y in els do Add(pv,[o,x,y]);od;od;
for x in els do Add(pv,[z,o,x]);od;Add(pv,[z,z,o]);
pts:=Filtered(pv,v->Sum(v,x->x^(q+1))=z);
blocks:=[];
for c in pv do
 B:=Filtered([1..Length(pts)],i->Sum([1..3],j->pts[i][j]*c[j])=z);
 if Length(B)=q+1 then AddSet(blocks,B);fi;
od;
need(Length(pts)=65 and Length(blocks)=208,"unital parameters");
need(ForAll(Combinations([1..65],2),P->Number(blocks,B->IsSubset(B,P))=1),"design pair uniqueness");
L:=blocks[1];p:=L[1];left:=Difference([1..65],L);right:=[];
for a in Difference(L,[p]) do for M in blocks do
 if a in M and M<>L then Add(right,[a,M]);fi;
od;od;
need(Length(left)=60 and Length(right)=60,"residue part sizes");
adj:=List([1..120],x->[]);
for i in [1..60] do for j in [1..60] do
 if left[i] in right[j][2] then Add(adj[i],j+60);Add(adj[j+60],i);fi;
od;od;
need(Set(List(adj,Length))=[4],"four-regular residue");
profiles:=[];girth:=120;
for root in [1..120] do
 dist:=List([1..120],x->-1);parent:=List([1..120],x->0);dist[root]:=0;queue:=[root];pos:=1;
 while pos<=Length(queue) do
  a:=queue[pos];pos:=pos+1;
  for b in adj[a] do
   if dist[b]=-1 then dist[b]:=dist[a]+1;parent[b]:=a;Add(queue,b);
   elif parent[a]<>b then girth:=Minimum(girth,dist[a]+dist[b]+1);fi;
  od;
 od;
 need(Length(queue)=120,"all-root connectivity");
 profile:=[];for d in [0..Maximum(dist)] do Add(profile,Number(dist,x->x=d));od;Add(profiles,profile);
od;
need(girth=8 and ForAll(profiles,P->Length(P)=7),"girth eight and diameter six");
Print("PHASE265_RESIDUE_PROFILES ",Set(profiles{[1..60]}),"  ",Set(profiles{[61..120]}),"\n");
need(Set(profiles{[1..60]})=[[1,4,12,36,44,20,3]],"point profile");
need(Set(profiles{[61..120]})=[[1,4,12,36,42,20,5]],"line profile");
# Any graph automorphism preserves distance profiles, so these distinct
# profiles exclude every part-interchanging automorphism.
Print("PHASE265_UH4_PASS points65 blocks208 residue120 edges240 degree4 girth8 diameter6 no_duality\n");
QUIT;
```

3. phase186_witt_derived_check.g

Role: required mathematical computation. Five actual native Witt designs, full groups, three coordinate-plane maps, all displayed stabilizer actions and extensions.

```
# Native Mathieu designs, exact full automorphism groups, and explicit
# derived-plane isomorphisms. This is finite GAP/GRAPE evidence, not Lean.
LoadPackage("design");;
LoadPackage("grape");;
SizeScreen([100000,24]);;
if not IsBound(P186_OUTPUT) then P186_OUTPUT:="/private/tmp/phase186_witt_witness.json";fi;
P186Require:=function(test,msg) if not test then Error(msg);fi;end;;
P186Models:=rec();;
P186Plane:=function()
 local F,raw,vecs,blocks,p,normal,linear,semilinear;
 F:=GF(4);raw:=Filtered(Tuples(AsList(F),3),x->not IsZero(x));
 vecs:=Set(List(raw,x->x/First(x,y->y<>Zero(F))));
 blocks:=Set(List(vecs,n->Filtered([1..21],i->Sum([1..3],j->n[j]*vecs[i][j])=Zero(F))));
 P186Require(Length(vecs)=21 and Length(blocks)=21,"plane size");
 P186Require(Set(List(blocks,Length))=[5],"plane line size");
 normal:=function(v,m) local w;w:=v*m;return w/First(w,x->x<>Zero(F));end;
 linear:=Action(SL(3,4),vecs,normal);
 semilinear:=Group(Concatenation(GeneratorsOfGroup(Action(GL(3,4),vecs,normal)),[PermList(List(vecs,v->Position(v
 ecs,List(v,x->x^2))))]));
 P186Require(Size(linear)=20160 and Size(semilinear)=120960,"coordinate collineation groups");
 return rec(vectors:=vecs,codes:=List(vecs,x->List(x,a->Position([Zero(F),One(F),Z(4),Z(4)^2],a)-1)),blocks:=bloc
 ks,design:=BlockDesign(21,blocks),linear:=linear,semilinear:=semilinear);
end;;
P186Stabilizers:=function(v,t,k,G,blocks,plane)
 local B,H,hom,N,C,complements,T,M,MC,Q,Z,j,R,rem,db,target,
       iso,pm,J,img,pointwise,pi,labels,expected,outer;
 B:=blocks[1];H:=Stabilizer(G,B,OnSets);
 hom:=ActionHomomorphism(H,B,OnPoints);N:=Kernel(hom);
 P186Require(Size(N)=16 and IsElementaryAbelian(N),"block kernel is not 2^4");
 P186Require(Image(hom)=AlternatingGroup(k),"block action not A_k");
 complements:=ComplementClassesRepresentatives(H,N);
 P186Require(Length(complements)>0,"no complement to block kernel");C:=complements[1];
 P186Require(Size(Intersection(C,N))=1 and Size(C)*Size(N)=Size(H),"bad block complement");
 T:=B{[1..t-1]};M:=Stabilizer(H,T,OnSets);MC:=Stabilizer(C,T,OnSets);
 P186Require(Size(Intersection(MC,N))=1 and Size(MC)*16=Size(M),"marked split failure");
 P186Require(Image(hom,M)=Stabilizer(AlternatingGroup(k),[1..t-1],OnSets),"marked alternating subset action");
 if v=22 then
  Q:=PCore(M,2);Z:=Centre(Q);
  P186Require(Size(Q)=64 and Size(Z)=4 and IsElementaryAbelian(Z),"M22 marked central 2^2");
  P186Require(IsElementaryAbelian(FactorGroup(Q,Z)),"M22 marked central quotient not 2^4");
  P186Require(IsomorphismGroups(FactorGroup(M,Q),SymmetricGroup(3))<>fail,"M22 marked quotient not S3");
  P186Require(Length(ComplementClassesRepresentatives(M,Q))>0,"M22 marked outer split");
 fi;
 labels:=[];
 for j in [1..t-2] do
  R:=[1..j];rem:=Difference([1..v],R);
  db:=Set(List(Filtered(blocks,B->IsSubset(B,R)),B->List(Difference(B,R),x->Position(rem,x))));
  if j=t-2 then target:=rec(design:=plane.design,native:=plane.linear,full:=plane.semilinear);
  else target:=P186Models.(String(v-j));fi;
  iso:=GraphIsomorphism(
   rec(graph:=IncidenceGraphSupportBlockDesign(BlockDesign(v-j,db)),colourClasses:=[[1..v-j],[v-j+1..v-j+Length(db
   )]]),
   rec(graph:=IncidenceGraphSupportBlockDesign(target.design),colourClasses:=[[1..v-j],[v-j+1..v-j+Length(db)]]));
  P186Require(iso<>fail,"ordinary derived model not isomorphic");
  pm:=PermList(List([1..v-j],i->i^iso));
  P186Require(Set(List(db,B->OnSets(B,pm)))=Set(target.design.blocks),"ordinary map block check");
  J:=Stabilizer(G,R,OnSets);pi:=ActionHomomorphism(J,rem,OnPoints);
  P186Require(Size(Kernel(pi))=1,"ordinary residual action has kernel");img:=Image(pi)^pm;
  pointwise:=Stabilizer(G,R,OnTuples);
  P186Require(Image(pi,pointwise)^pm=target.native,"pointwise ordinary action not native group");
  if j=1 then P186Require(img=target.native,"point ordinary stabilizer");
  elif j=t-2 then
   P186Require(IsSubgroup(target.full,img) and Size(img)=20160*Factorial(j),"projective ordinary outer group");
   if j=2 then
    outer:=FactorGroup(img,target.native);
    P186Require(Size(outer)=2 and Length(ComplementClassesRepresentatives(img,target.native))>0,"projective
    index-two extension not split");
   else P186Require(img=target.full,"full projective semilinear action");fi;
  else P186Require(img=target.full,"ordinary full derived action");fi;
```

```
  Add(labels,rec(j:=j,order:=Size(J),point_map:=List([1..v-j],i->i^pm)));
 od;
 Print("P186_FIGURE3_STABILIZERS ",rec(v:=v,block_order:=Size(H),block_kernel_order:=Size(N),block_complement_ord
 er:=Size(C),marked_order:=Size(M),marked_complement_order:=Size(MC),ordinary:=labels),"\n");
 AppendTo(P186_OUTPUT,",\"block_order\":",Size(H),",\"block_kernel_order\":",Size(N),",\"block_complement_order\"
 :",Size(C),",\"marked_order\":",Size(M),",\"marked_complement_order\":",Size(MC),",\"ordinary_orders\":",List(la
 bels,x->x.order),",\"ordinary_point_maps\":",List(labels,x->x.point_map),",\"block_kernel_generators\":",List(Ge
 neratorsOfGroup(N),g->List([1..v],i->i^g)),",\"block_complement_generators\":",List(GeneratorsOfGroup(C),g->List
 ([1..v],i->i^g)));
end;;
P186Run:=function(v,t,k,plane)
 local G,A,H,orbs,small,seed,blocks,expected,allorbs,design,full,
       R,remaining,derived,ddesign,iso,pmap,mapped,B,g,counts,record,orbit_sizes;
 G:=MathieuGroup(v);A:=[1..t];expected:=Binomial(v,t)/Binomial(k,t);
 if v<20 then
  allorbs:=OrbitsDomain(G,Combinations([1..v],k),OnSets);
  small:=Filtered(allorbs,o->Length(o)=expected);
  P186Require(Length(small)=1,"small Witt orbit not unique");
  P186Require(ForAll(allorbs,o->Length(o)>=expected),"possible mixed block orbit");
  orbit_sizes:=SortedList(List(allorbs,Length));
  blocks:=Set(small[1]);
 else
  H:=Stabilizer(G,A,OnTuples);orbs:=Orbits(H,Difference([1..v],A));
  small:=Filtered(orbs,o->Length(o)=k-t);
  P186Require(Length(small)=1 and ForAll(orbs,o->Length(o)>=k-t),"large seed not unique");
  orbit_sizes:=SortedList(List(orbs,Length));
  seed:=Union(A,small[1]);blocks:=Set(Orbit(G,seed,OnSets));
 fi;
 P186Require(Length(blocks)=expected,"block count");
 counts:=List(Combinations([1..v],t),X->Number(blocks,B->IsSubset(B,X)));
 P186Require(Set(counts)=[1],"not a Steiner design");
 # Canonical group identification is a retained GAP-library input. The full
 # design automorphism computation itself is performed afresh by DESIGN.
 design:=BlockDesign(v,blocks);full:=AutGroupBlockDesign(design);
 P186Require(IsSubgroup(full,G),"native group not in full group");
 for g in GeneratorsOfGroup(full) do
  P186Require(Set(List(blocks,B->OnSets(B,g)))=blocks,"bad full-group generator");
 od;
 if v=22 then
  P186Require(Size(full)=2*Size(G),"M22 full group order");
  P186Require(Length(ComplementClassesRepresentatives(full,G))>0,"M22 full index-two extension not split");
 else P186Require(full=G,"full group differs from native Mathieu group");fi;
 P186Models.(String(v)):=rec(design:=design,native:=G,full:=full);
 Print("P186_FULL_AUT ",rec(v:=v,t:=t,k:=k,blocks:=Length(blocks),tested_subsets:=Length(counts),native_order:=Si
 ze(G),full_order:=Size(full)),"\n");
 if v<>11 then AppendTo(P186_OUTPUT,",\n");fi;
 AppendTo(P186_OUTPUT,"{\"v\":",v,",\"t\":",t,",\"k\":",k,",\"blocks\":",blocks,",\"orbit_sizes\":",orbit_sizes,"
 ,\"native_order\":",Size(G),",\"full_order\":",Size(full),",\"native_generators\":",List(GeneratorsOfGroup(G),g-
 >List([1..v],i->i^g)),",\"full_generators\":",List(GeneratorsOfGroup(full),g->List([1..v],i->i^g)));
 if v>20 then
  R:=[1..t-2];remaining:=Difference([1..v],R);
  derived:=Set(List(Filtered(blocks,B->IsSubset(B,R)),B->List(Difference(B,R),p->Position(remaining,p))));
  P186Require(Length(remaining)=21 and Length(derived)=21,"derived sizes");
  ddesign:=BlockDesign(21,derived);
  iso:=GraphIsomorphism(
   rec(graph:=IncidenceGraphSupportBlockDesign(ddesign),colourClasses:=[[1..21],[22..42]]),
   rec(graph:=IncidenceGraphSupportBlockDesign(plane.design),colourClasses:=[[1..21],[22..42]]));
  P186Require(iso<>fail,"derived design is not PG(2,4)");
  pmap:=List([1..21],i->i^iso);
  P186Require(Set(pmap)=[1..21],"not a point bijection");
  mapped:=Set(List(derived,B->Set(List(B,i->pmap[i]))));
  P186Require(mapped=plane.blocks,"explicit incidence map failure");
  # Check every one of the 441 point/line incidences, including non-incidences.
  P186Require(ForAll(derived,B->ForAll([1..21],i->(i in B)=(pmap[i] in
  plane.blocks[Position(plane.blocks,Set(List(B,j->pmap[j])))]))),"incidence equivalence");
  Print("P186_DERIVED_PLANE ",rec(v:=v,derived_at:=R,remaining_points:=remaining,point_map:=pmap,derived_blocks:=d
  erived,target_blocks:=plane.blocks),"\n");
  AppendTo(P186_OUTPUT,",\"derived_at\":[",JoinStringsWithSeparator(List(R,String),","),"],\"remaining_points\":"
  ,remaining,",\"point_map\":",pmap,",\"derived_blocks\":",derived);
  P186Stabilizers(v,t,k,G,blocks,plane);
 fi;
 AppendTo(P186_OUTPUT,"}");
 Print("P186_WITT_MODEL
 ",rec(v:=v,blocks:=blocks,generators:=GeneratorsOfGroup(G),full_generators:=GeneratorsOfGroup(full)),"\n");
end;;
```

```
P186PG:=P186Plane();;
PrintTo(P186_OUTPUT,"{\"gap_version\":\"",GAPInfo.Version,"\",\"design_version\":\"",PackageInfo("design")[1].Ver
sion,"\",\"grape_version\":\"",PackageInfo("grape")[1].Version,"\",\"field_polynomial\":\"z^2+z+1\",\"plane_vecto
rs\":",P186PG.codes,",\"plane_blocks\":",P186PG.blocks,",\"cases\":[\n");
Print("P186_PG4_VECTORS ",P186PG.vectors,"\n");
P186Run(11,4,5,P186PG);
P186Run(12,5,6,P186PG);
P186Run(22,3,6,P186PG);
P186Run(23,4,7,P186PG);
P186Run(24,5,8,P186PG);
AppendTo(P186_OUTPUT,"\n],\"projective_labels\":{\"v\":21");
P186Stabilizers(21,2,5,P186PG.linear,P186PG.blocks,P186PG);
AppendTo(P186_OUTPUT,"}}\n");
Print("P186_WITT_DERIVED_AND_FULL_AUT_PASS\n");
QUIT;
```

4. phase182_mathieu_fibers.g

Role: required mathematical computation. All three large Mathieu chamber extension fibers, with actual block uniqueness and group action.

```
# Exact extension-fiber checks for the large Mathieu Steiner systems.
# Named Mathieu actions are retained inputs. This does not reprove their
# classification, uniqueness, or the uniform residue theorem.
P182Require := function(b,msg) if not b then Error(msg); fi; end;
P182Check := function(v,t,k)
  local G,pts,A,H,orbs,small,B,blocks,counts,T,B0,alternatives,K,B1,K1,
        tail,orbit1,orbit2,expected,chambers;
  G:=MathieuGroup(v); pts:=[1..v]; A:=[1..t];
  H:=Stabilizer(G,A,OnTuples);
  orbs:=Orbits(H,Difference(pts,A));
  small:=Filtered(orbs,o->Length(o)=k-t);
  P182Require(Length(small)=1,"ambiguous seed block");
  B:=Union(A,small[1]);
  blocks:=Set(Orbit(G,B,OnSets));
  expected:=Binomial(v,t)/Binomial(k,t);
  P182Require(Length(blocks)=expected,"wrong block orbit size");
  # Verify all t-subsets explicitly, rather than infer a Steiner system
  # from block size and group transitivity alone.
  counts:=List(Combinations(pts,t),X->Number(blocks,C->IsSubset(C,X)));
  P182Require(Set(counts)=[1],"Steiner condition failure");
  T:=[1..t-1];
  B0:=First(blocks,C->IsSubset(C,T));
  alternatives:=Filtered(blocks,C->IsSubset(C,T) and C<>B0);
  K:=Intersection(Stabilizer(G,T,OnTuples),Stabilizer(G,B0,OnSets));
  P182Require(Index(G,K)=Product([0..t-2],i->v-i)*5,
              "base-flag orbit does not exhaust all base flags");
  P182Require(Length(alternatives)=4,"wrong alternative-block count");
  B1:=alternatives[1]; orbit1:=Set(Orbit(K,B1,OnSets));
  P182Require(orbit1=Set(alternatives),"alternative-block orbit incomplete");
  K1:=Stabilizer(K,B1,OnSets); tail:=Difference(B1,T);
  orbit2:=Set(Orbit(K1,tail[1]));
  P182Require(orbit2=Set(tail) and Length(tail)=4,"tail orbit incomplete");
  chambers:=Product([0..t-1],i->v-i)*4;
  P182Require(Size(Stabilizer(K1,tail[1]))=12,"wrong chamber stabilizer");
  P182Require(Size(G)=12*chambers,"wrong chamber orbit count");
  Print("P182_MATHIEU ",rec(v:=v,t:=t,k:=k,group_order:=Size(G),
    block_count:=Length(blocks),tested_subsets:=Length(counts),
    base_stabilizer_order:=Size(K),alternative_orbit:=Length(orbit1),
    tail_stabilizer_order:=Size(K1),tail_orbit:=Length(orbit2),
    chambers:=chambers,chamber_stabilizer:=12),"\n");
end;
P182Check(22,3,6);
P182Check(23,4,7);
P182Check(24,5,8);
Print("P182_MATHIEU_EXTENSION_FIBERS_PASS\n");
QUIT;
```

5. phase186_witt_validate.py

Role: required witness verification. Independent arithmetic/Steiner/incidence validation of native program witness. Does not recompute full group completeness.

```
#!/usr/bin/env python3
"""Independent finite-incidence validation; no group order oracle is implemented."""
import argparse
import copy
from collections import Counter
from itertools import combinations, product
import json
from math import comb
from pathlib import Path


def require(test, message):
    if not test:
        raise ValueError(message)


def mul4(a, b):
    """Coefficient bits in F2[z]/(z*z+z+1): 0,1,z,z+1."""
    c0 = (a & 1) * (b & 1)
    c1 = ((a & 1) * (b >> 1)) ^ ((a >> 1) * (b & 1))
    c2 = (a >> 1) * (b >> 1)
    return (c0 ^ c2) | ((c1 ^ c2) << 1)


def dot4(a, b):
    return mul4(a[0], b[0]) ^ mul4(a[1], b[1]) ^ mul4(a[2], b[2])


def canonical4(a):
    first = next(x for x in a if x)
    inverse = next(x for x in range(1, 4) if mul4(first, x) == 1)
    return tuple(mul4(x, inverse) for x in a)


def validate(data):
    require(data['field_polynomial'] == 'z^2+z+1', 'field polynomial')
    vectors = [tuple(v) for v in data['plane_vectors']]
    expected_vectors = {canonical4(v) for v in product(range(4), repeat=3) if any(v)}
    require(len(vectors) == 21 and set(vectors) == expected_vectors, 'projective points')
    target = {tuple(i + 1 for i, v in enumerate(vectors) if dot4(n, v) == 0)
              for n in vectors}
    require(len(target) == 21 and {len(b) for b in target} == {5}, 'coordinate lines')
    require(target == {tuple(b) for b in data['plane_blocks']}, 'GAP coordinate plane')
    reports = []
    require([c['v'] for c in data['cases']] == [11, 12, 22, 23, 24], 'case completeness')
    for case in data['cases']:
        v, t, k = case['v'], case['t'], case['k']
        expected = {11:(4,5), 12:(5,6), 22:(3,6), 23:(4,7), 24:(5,8)}[v]
        require((t, k) == expected, 'design parameters')
        blocks = [tuple(b) for b in case['blocks']]
        require(len(blocks) == len(set(blocks)) == comb(v,t)//comb(k,t), 'block count')
        require(all(len(b) == k and tuple(sorted(set(b))) == b and
                    all(1 <= x <= v for x in b) for b in blocks), 'invalid block')
        counts = Counter(s for b in blocks for s in combinations(b, t))
        require(len(counts) == comb(v,t) and set(counts.values()) == {1}, 'Steiner property')
        for key in ('native_generators', 'full_generators'):
            for g in case[key]:
                require(sorted(g) == list(range(1,v+1)), 'not a permutation')
                require({tuple(sorted(g[x-1] for x in b)) for b in blocks} == set(blocks),
                        'group generator violates design')
        # Reported orders depend on the GAP group and DESIGN/GRAPE calculations;
        # comparing the numbers here is only a schema/claim check.
        native = {11:7920,12:95040,22:443520,23:10200960,24:244823040}[v]
        require(case['native_order'] == native, 'native group order claim')
        require(case['full_order'] == native * (2 if v == 22 else 1), 'full order claim')
        if v > 20:
            removed = list(range(1,t-1))
            remaining = list(range(t-1,v+1))
            require(case['derived_at'] == removed and case['remaining_points'] == remaining,
```

```
                'derived point convention')
            derived = {tuple(remaining.index(x)+1 for x in b if x not in removed)
                       for b in blocks if set(removed) <= set(b)}
            require(derived == {tuple(b) for b in case['derived_blocks']}, 'derived blocks')
            p = case['point_map']
            require(sorted(p) == list(range(1,22)), 'derived point map')
            mapped = {tuple(sorted(p[x-1] for x in b)) for b in derived}
            require(mapped == target, 'derived map does not preserve coordinate incidence')
            for b in derived:
                line = tuple(sorted(p[x-1] for x in b))
                for point in range(1,22):
                    require((point in b) == (p[point-1] in line), 'incidence iff')
        reports.append({'v':v,'blocks':len(blocks),'Steiner_subsets':len(counts),
                        'coordinate_incidence_pairs':441 if v>20 else 0})
    return reports


def negative_controls(data):
    bad_block = copy.deepcopy(data)
    b = bad_block['cases'][0]['blocks'][0]
    b[-1] = next(x for x in range(1,12) if x not in b)
    b.sort()
    bad_map = copy.deepcopy(data)
    p = bad_map['cases'][2]['point_map']
    p[0], p[1] = p[1], p[0]
    results = []
    for label, corrupted in [('alter_one_Witt_block',bad_block),
                             ('swap_two_derived_images',bad_map)]:
        try:
            validate(corrupted)
        except ValueError as error:
            results.append({'control':label,'status':'REJECTED','reason':str(error)})
        else:
            raise ValueError('negative control accepted: ' + label)
    return results


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('witness', type=Path)
    parser.add_argument('--output',type=Path)
    args = parser.parse_args()
    data = json.loads(args.witness.read_text())
    result = {'status':'PASS','cases':validate(data),'negative_controls':negative_controls(data),
              'scope':'Finite incidence arithmetic and generator preservation. Full automorphism completeness and
              group orders remain GAP/DESIGN/GRAPE computations; no Lean certificate.'}
    text = json.dumps(result,indent=2)+'\n'
    if args.output:
        args.output.write_text(text)
    print(text,end='')


if __name__ == '__main__':
    main()
```

6. phase186_witt_lean_generate.py

Role: required certificate bridge. Reconstructs the three explicit finite Lean tables from the native design witness, compared byte-for-byte with frozen formal source.

```
#!/usr/bin/env python3
"""Emit an ordinary kernel-checked Lean certificate from explicit finite maps."""
import argparse
import json
from pathlib import Path
from phase186_witt_validate import dot4


def vec(items):
    return '![' + ', '.join(items) + ']'
```

```
def generate(data):
    field = ['(0, 0)', '(1, 0)', '(0, 1)', '(1, 1)']
    # GAP sorts its set of line supports, whereas Lean indexes linear forms
    # in coordinate-vector order. Bind the two conventions explicitly.
    coordinate_lines = [[i+1 for i,v in enumerate(data['plane_vectors']) if dot4(n,v)==0]
                        for n in data['plane_vectors']]
    out = ['import DeletedLineFourResidue', '',
           '/-! Explicit finite derived-plane incidence maps. The original Witt',
           'designs and their derivation remain GAP-checked inputs; this file',
           'checks every incidence against literal F4 coordinate arithmetic. -/', '',
           'namespace MathCompression.WittDerivedPlanes',
           'open MathCompression.DeletedLineFourResidue',
           'set_option maxRecDepth 1000000',
           'set_option maxHeartbeats 0', '',
           'def coordinate : Fin 21 → Fin 3 → F4 :=',
           '  ' + vec([vec([field[x] for x in v]) for v in data['plane_vectors']]), '',
           'def coordinateIncident (p l : Fin 21) : Prop :=',
           '  mul4 (coordinate p 0) (coordinate l 0) +',
           '  mul4 (coordinate p 1) (coordinate l 1) +',
           '  mul4 (coordinate p 2) (coordinate l 2) = 0', '',
           'instance (p l : Fin 21) : Decidable (coordinateIncident p l) :=',
           '  inferInstanceAs (Decidable (_ = _))', '',
           'structure IncidenceIso (source : Fin 21 → Fin 21 → Prop) where',
           '  points : Fin 21 ≃ Fin 21',
           '  lines : Fin 21 ≃ Fin 21',
           '  preserves : ∀ p l, source p l ↔ coordinateIncident (points p) (lines l)', '']
    for case in data['cases']:
        if case['v'] < 20:
            continue
        suffix = str(case['v'])
        lines = case['derived_blocks']
        pm = case['point_map']
        lm = [coordinate_lines.index(sorted(pm[x-1] for x in b)) for b in lines]
        out += [f'def nativeLines{suffix} : Fin 21 → List (Fin 21) :=',
                '  ' + vec(['[' + ', '.join(str(x-1) for x in b) + ']' for b in lines]), '',
                f'def pointMap{suffix} : Fin 21 → Fin 21 :=',
                '  ' + vec([str(x-1) for x in pm]), '',
                f'def lineMap{suffix} : Fin 21 → Fin 21 :=',
                '  ' + vec([str(x) for x in lm]), '',
                f'theorem maps{suffix}_bijective : Function.Bijective pointMap{suffix} ∧',
                f'    Function.Bijective lineMap{suffix} := by decide', '',
                f'theorem incidence{suffix} : ∀ p l : Fin 21,',
              f'    p ∈ nativeLines{suffix} l ↔ coordinateIncident (pointMap{suffix} p) (lineMap{suffix} l) := by',
                '  decide', '',
                f'noncomputable def iso{suffix} : IncidenceIso (fun p l => p ∈ nativeLines{suffix} l) where',
                f'  points := Equiv.ofBijective pointMap{suffix} maps{suffix}_bijective.1',
                f'  lines := Equiv.ofBijective lineMap{suffix} maps{suffix}_bijective.2',
                f'  preserves := incidence{suffix}', '',
                f'#print axioms maps{suffix}_bijective',
                f'#print axioms incidence{suffix}',
                f'#print axioms iso{suffix}', '']
    out += ['end MathCompression.WittDerivedPlanes', '']
    return '\n'.join(out)


def main():
    p = argparse.ArgumentParser()
    p.add_argument('witness',type=Path)
    p.add_argument('output',type=Path)
    args = p.parse_args()
    args.output.write_text(generate(json.loads(args.witness.read_text())))


if __name__ == '__main__':
    main()
```

7. phase265_frame_control.g

Role: validation. Concrete F9 semilinear counterexample and missing-SL hypothesis control; uniform proof remains written.

```
# An affine frame-transitive semilinear group need not contain GL(2,q).
```

```
# This is an exact finite source-clause diagnostic, not the uniform proof.
Need := function(b,s) if not b then Error(s); fi; end;
F:=GF(9); z:=Z(9); o:=One(F); zero:=Zero(F);
V:=Tuples(Elements(F),2);
ToPerm:=function(A,e)
 return PermList(List(V,v->Position(V,List(v,x->x^(3^e))*A)));
end;
I:=IdentityMat(2,F);
slm:=GeneratorsOfGroup(SL(2,9));
slp:=List(slm,A->ToPerm(A,0));
squares:=ToPerm([[z^2,zero],[zero,o]],0);
twist:=ToPerm([[z,zero],[zero,o]],1);
H:=Group(Concatenation(slp,[squares,twist]));
SLP:=Group(slp); GLP:=Group(List(GeneratorsOfGroup(GL(2,9)),A->ToPerm(A,0)));
Need(Size(SLP)=720,"SL order");
Need(Size(H)=5760 and Size(GLP)=5760,"equal frame-group orders");
Need(IsSubgroup(H,SLP),"SL inclusion");
Need(not IsSubgroup(H,GLP),"literal GL containment fails");
frame:=[Position(V,[o,zero]),Position(V,[zero,o])];
Need(Size(Stabilizer(H,frame,OnTuples))=1,"regular frame action");
Need(Length(Orbit(H,frame,OnTuples))=5760,"all independent frames");
translations:=List([[o,zero],[zero,o],[z,zero],[zero,z]],
 w->PermList(List(V,v->Position(V,v+w))));
G:=Group(Concatenation(GeneratorsOfGroup(H),translations));
Need(Size(G)=466560,"affine group order");
origin:=Position(V,[zero,zero]);
Need(Size(Stabilizer(G,Concatenation([origin],frame),OnTuples))=1,"triangle stabilizer");
Need(Size(G)=81*80*72,"full ordered triangle orbit size");
lines:=Set(Concatenation(List(V,v->List(Filtered(V,w->w<>[zero,zero]),
 w->Set(List(Elements(F),a->Position(V,v+a*w)))))));
Need(Length(lines)=90,"affine line count");
Need(ForAll(GeneratorsOfGroup(G),g->ForAll(lines,L->OnSets(L,g) in lines)),"literal line preservation");
Need(15*14*13>Size(AlternatingGroup(7)),"A7 cannot be 3-transitive on15 points");
# Removing the SL-fiber hypothesis makes determinant transitivity insufficient.
F3:=GF(3);e3:=One(F3);z3:=Zero(F3);V3:=Tuples(Elements(F3),2);
diag3:=Group(List([[[ -e3,z3],[z3,e3]],[[e3,z3],[z3,-e3]]],
 A->PermList(List(V3,v->Position(V3,v*A)))));
frame3:=[Position(V3,[e3,z3]),Position(V3,[z3,e3])];
Need(Size(diag3)=4 and Length(Orbit(diag3,frame3,OnTuples))=4 and 4<48,"determinant-only weakening rejected");
Print("PHASE265_NO_SL_CONTROL_PASS determinant_image2 frame_orbit4_not48\n");
Print("PHASE265_FRAME_CONTROL_PASS H5760 G466560 lines90 full_triangle_orbit GL_not_contained
A7_degree15_excluded\n");
QUIT;
```

8. phase265_plane_models.g

Role: validation. Nine literal plane/circle models check common-neighbor formulas and fixed small cases.

```
# Literal flag relations and complete deleted-pencil residue graphs.
Need:=function(b,s) if not b then Error(s);fi;end;
Normalize:=function(v)
 local a;
 a:=First(v,x->not IsZero(x));Need(a<>fail,"nonzero projective vector");return v/a;
end;
Affine:=function(d,q)
 local K,V,dirs,v,w,lines;
 K:=GF(q);V:=Tuples(Elements(K),d);dirs:=Set(List(Filtered(V,v->not IsZero(v)),Normalize));lines:=[];
 for v in V do for w in dirs do AddSet(lines,Set(List(Elements(K),a->Position(V,v+a*w))));od;od;
 return rec(name:=Concatenation("AG",String(d),",",String(q)),v:=Length(V),blocks:=lines);
end;
Projective:=function(d,q)
 local K,V,lines,i,j,L,a,b;
 K:=GF(q);V:=Set(List(Filtered(Tuples(Elements(K),d+1),v->not IsZero(v)),Normalize));lines:=[];
 for i in [1..Length(V)] do for j in [i+1..Length(V)] do
  L:=[];
  for a in Elements(K) do for b in Elements(K) do
   if not (IsZero(a) and IsZero(b)) then AddSet(L,Position(V,Normalize(a*V[i]+b*V[j])));fi;
  od;od;AddSet(lines,L);
 od;od;
 return rec(name:=Concatenation("PG",String(d),",",String(q)),v:=Length(V),blocks:=lines);
end;
```

```
Check:=function(S,expectedComponents,expectedGirth,expectedDiameter)
 local v,blocks,k,r,flags,p,L,D,E,i,j,cs,cp,want,outside,right,a,M,adj,root,dist,parent,queue,pos,x,y,girth,diam,
 comps,seen,profile;
 v:=S.v;blocks:=S.blocks;k:=Length(blocks[1]);r:=(v-1)/(k-1);
 Need(ForAll(blocks,L->Length(L)=k),"line uniformity");
 Need(ForAll(Combinations([1..v],2),P->Number(blocks,L->IsSubset(L,P))=1),"unique joining line");
 Need(ForAll([1..v],p->Number(blocks,L->p in L)=r),"point uniformity");
 flags:=[];for p in [1..v] do for L in blocks do if p in L then Add(flags,[p,L]);fi;od;od;
 D:=List(flags,f->Filtered([1..Length(flags)],j->f[1] in flags[j][2] and not flags[j][1] in f[2]));
 E:=List([1..Length(flags)],j->Filtered([1..Length(flags)],i->j in D[i]));
 Need(ForAll(D,row->Length(row)=v-k),"literal directed degrees");
 for i in [1..Length(flags)] do for j in [i+1..Length(flags)] do
  cs:=Length(Intersection(D[i],D[j]));cp:=Length(Intersection(E[i],E[j]));
  if flags[i][1]=flags[j][1] then Need(cs=(r-2)*(k-1),"same-point signature");else Need(cs in
  [0,k-2],"different-point signature");fi;
  if flags[i][2]=flags[j][2] then Need(cp=(k-2)*(r-1),"same-line signature");else Need(cp in
  [0,r-2],"different-line signature");fi;
 od;od;
 L:=blocks[1];p:=L[1];outside:=Difference([1..v],L);right:=[];
 for a in Difference(L,[p]) do for M in blocks do if a in M and M<>L then Add(right,[a,M]);fi;od;od;
 adj:=List([1..Length(outside)+Length(right)],i->[]);
 for i in [1..Length(outside)] do for j in [1..Length(right)] do
  if outside[i] in right[j][2] then Add(adj[i],Length(outside)+j);Add(adj[Length(outside)+j],i);fi;
 od;od;
 Need(Set(List(adj,Length))=[k-1],"every residue degree");
 girth:=Length(adj)+1;diam:=0;comps:=[];seen:=[];
 for root in [1..Length(adj)] do
  dist:=List(adj,x->-1);parent:=List(adj,x->0);dist[root]:=0;queue:=[root];pos:=1;
  while pos<=Length(queue) do
   x:=queue[pos];pos:=pos+1;
   for y in adj[x] do
    if dist[y]=-1 then dist[y]:=dist[x]+1;parent[y]:=x;Add(queue,y);
    elif parent[x]<>y then girth:=Minimum(girth,dist[x]+dist[y]+1);fi;
   od;
  od;
  diam:=Maximum(diam,Maximum(dist));
  if not root in seen then Add(comps,Length(queue));UniteSet(seen,Set(queue));fi;
 od;
 Need(Length(comps)=expectedComponents and girth=expectedGirth and diam=expectedDiameter,"entire residue
 invariants");
 Print("PHASE265_PLANE_PASS ",S.name," flags",Length(flags),"
 all_signature_pairs",Length(flags)*(Length(flags)-1)/2,
 " residue_components",comps," girth",girth," diameter",diam,"\n");
end;
Check(Projective(2,2),1,8,4);
Check(Projective(2,3),1,6,4);
Check(Projective(2,4),1,6,4);
Check(Projective(2,5),1,6,4);
Check(Affine(2,3),1,12,6);
Check(Affine(2,4),1,6,4);
Check(Affine(2,5),1,6,4);
Check(Projective(3,2),3,8,4);
Check(Affine(3,3),4,12,6);
Print("PHASE265_ALL_NINE_MODELS_PASS\n");
QUIT;
```

9. phase186_hermitian_labels.g

Role: validation. All nine printed q values in the proven root-coset incidence model; not proof for all q.

```
# Finite audit of the source-isomorphic norm-shell/root-coset residue model.
P186HRequire:=function(test,msg)if not test then Error(msg);fi;end;;
P186HDist:=function(adj,root)
 local ds,queue,pos,x,y,par,cycle;
 ds:=List(adj,x->-1);par:=List(adj,x->0);ds[root]:=0;queue:=[root];pos:=1;cycle:=infinity;
 while pos<=Length(queue) do
  x:=queue[pos];pos:=pos+1;
  for y in adj[x] do
   if ds[y]=-1 then ds[y]:=ds[x]+1;par[y]:=x;Add(queue,y);
   elif par[x]<>y and par[y]<>x then cycle:=Minimum(cycle,ds[x]+ds[y]+1);fi;
  od;
 od;
 P186HRequire(not -1 in ds,"disconnected root-coset incidence graph");
```

```
 return [Maximum(ds),cycle];
end;;
P186HCheck:=function(q)
 local F,G,E,U,roots,flags,H,C,n,m,adj,i,j,dp,dl,expected;
 F:=GF(q);G:=SL(2,q);E:=Elements(G);
 U:=Group(List(Elements(F),t->[[One(F),t],[Zero(F),One(F)]]));
 roots:=Orbit(G,U,OnPoints);
 P186HRequire(Length(roots)=q+1 and ForAll(roots,H->Size(H)=q),"root groups");
 flags:=[];
 for H in roots do
  for C in RightCosets(G,H) do Add(flags,Set(List(AsList(C),g->Position(E,g))));od;
 od;
 n:=Length(E);m:=Length(flags);
 P186HRequire(n=q*(q^2-1) and m=(q+1)*(q^2-1),"residue type counts");
 P186HRequire(Length(Set(flags))=m,"duplicate flag supports");
 adj:=List([1..n+m],i->[]);
 for j in [1..m] do for i in flags[j] do Add(adj[i],n+j);Add(adj[n+j],i);od;od;
 P186HRequire(Set(List(adj{[1..n]},Length))=[q+1] and Set(List(adj{[n+1..n+m]},Length))=[q],"residue degrees");
 dp:=P186HDist(adj,1);dl:=P186HDist(adj,n+1);
 expected:=6;if IsEvenInt(q) then expected:=5;fi;
 P186HRequire(dp[1]=expected and dl[1]=6 and dp[2]=8 and dl[2]=8,"diameter or cycle length");
 Print("P186_HERMITIAN_LABEL ",rec(q:=q,points:=n,flags:=m,edges:=Sum(adj,Length)/2,point_diameter:=dp[1],flag_di
 ameter:=dl[1],girth:=dp[2]),"\n");
end;;
for q in [3,4,5,7,8,9,11,13,16] do P186HCheck(q);od;
Print("P186_HERMITIAN_LABELS_PASS\n");
QUIT;
```

10. phase182_deleted_line_check.py

Role: validation. Literal complete incidence and chamber actions at q4/q8; uniform full-group theorem remains written.

```
#!/usr/bin/env python3
"""Check literal L092 Construction 3 on the deleted-line affine geometries.

This checks every flag and every rank-at-least-two residue, and constructs
explicit incidence-preserving permutations. It does not compute the full
automorphism group or supply a Lean proof of the uniform classification.
"""
import argparse
from collections import Counter, deque
import hashlib
from itertools import combinations
import json
from pathlib import Path


def require(test, message):
    if not test:
        raise RuntimeError(message)


class Field:
    def __init__(self, q):
        require(q in (4, 8), "only the two stated fields are implemented")
        self.q = q
        self.modulus = {4: 0b111, 8: 0b1011}[q]

    def mul(self, a, b):
        result = 0
        while b:
            if b & 1:
                result ^= a
            b >>= 1
            a <<= 1
            if a & self.q:
                a ^= self.modulus
        return result

    def validate(self):
        Q = range(self.q)
```

```
        for a in Q:
            require(self.mul(a, 1) == a, "identity")
            if a:
                require(sum(self.mul(a, b) == 1 for b in Q) == 1, "inverse")
            for b in Q:
                require(self.mul(a, b) == self.mul(b, a), "commutativity")
                for c in Q:
                    require(self.mul(a, b ^ c) == self.mul(a, b) ^ self.mul(a, c),
                            "distributivity")
                    require(self.mul(self.mul(a, b), c) == self.mul(a, self.mul(b, c)),
                            "associativity")


def components(vertices, neighbours):
    remaining = set(vertices)
    result = []
    while remaining:
        root = min(remaining)
        seen, queue = {root}, deque([root])
        remaining.remove(root)
        while queue:
            current = queue.popleft()
            new = neighbours[current] & remaining
            seen.update(new)
            remaining.difference_update(new)
            queue.extend(sorted(new))
        result.append(sorted(seen))
    return result


def check(q):
    field = Field(q)
    field.validate()
    mul = field.mul
    points = [(x, y) for x in range(q) for y in range(1, q)]
    lines = [(a, b) for a in range(q) for b in range(q)]
    line_sets = [frozenset(i for i, (x, y) in enumerate(points) if x == mul(a, y) ^ b)
                 for a, b in lines]
    require(all(len(M) == q-1 for M in line_sets), "wrong source line size")
    require(all(sum(p in M for M in line_sets) == q for p in range(len(points))),
            "wrong source point degree")
    require(all(len(M & N) <= 1 for M, N in combinations(line_sets, 2)),
            "source is not a partial linear space")
    source_flags = [(p, m) for p in range(len(points)) for m, M in enumerate(line_sets)
                    if p in M]
    objects = ([(0, p) for p in range(len(points))]
               + [(1, m) for m in range(len(lines))]
               + [(2, flag) for flag in source_flags])
    bytype = [set(i for i, (t, _) in enumerate(objects) if t == j) for j in range(3)]
    index = {obj: i for i, obj in enumerate(objects)}
    neighbours = [set() for obj in objects]

    def incident(i, j):
        ti, x = objects[i]
        tj, y = objects[j]
        if ti > tj:
            return incident(j, i)
        if ti == tj:
            return False
        if (ti, tj) == (0, 1):
            return x not in line_sets[y]
        if (ti, tj) == (0, 2):
            p, m = y
            return x != p and x in line_sets[m]
        if (ti, tj) == (1, 2):
            p, m = y
            return line_sets[x] & line_sets[m] == {p}
        raise RuntimeError("missing incidence rule")

    for i, j in combinations(range(len(objects)), 2):
        if incident(i, j):
            neighbours[i].add(j)
            neighbours[j].add(i)

    allflags = [[] for _ in range(4)]
```

```
def enumerate_flags(prefix, choices):
    allflags[len(prefix)].append(tuple(prefix))
    for i in sorted(choices):
        enumerate_flags(prefix+[i], {j for j in choices & neighbours[i] if j > i})

enumerate_flags([], set(range(len(objects))))
chambers = set(allflags[3])
extended = set()
for C in chambers:
    for k in range(4):
        extended.update(combinations(C, k))
require(all(F in extended for layer in allflags for F in layer),
        "literal construction has a nonextendable flag")
panel_counts = Counter(len(neighbours[i] & neighbours[j]) for i, j in allflags[2])
require(min(panel_counts) >= 2, "literal construction is not firm")
require(len(chambers) == q*q*(q-1)**2*(q-2), "wrong chamber count")
whole_components = components(range(len(objects)), neighbours)
require(len(whole_components) == 1, "whole incidence graph disconnected")
residue_patterns = [Counter() for _ in range(3)]
disconnected_examples = []
for i, (t, obj) in enumerate(objects):
    pieces = components(neighbours[i], neighbours)
    signature = tuple(sorted(len(C) for C in pieces))
    residue_patterns[t][signature] += 1
    if len(pieces) > 1 and not disconnected_examples:
        disconnected_examples.append({'object':objects[i],
            'components':[[objects[j] for j in C] for C in pieces],
            'vertex_degrees':sorted(len(neighbours[j] & neighbours[i])
                                    for j in neighbours[i])})

# Lines (a,b) of the original structure are points of the dual net.
# Its lines are the sets ay+b=x with y nonzero. This identification
# allows an independently specified group action on the same incidence.
dual_lines = [frozenset(m for m, M in enumerate(line_sets) if p in M)
              for p in range(len(points))]
dual_line_index = {M:p for p, M in enumerate(dual_lines)}
line_index = {L:i for i, L in enumerate(lines)}
basis = [1 << k for k in range(q.bit_length()-1)]
dual_generators = []
for e in basis:
    dual_generators.append((f'translate_first_{e}', [(a ^ e, b) for a, b in lines]))
    dual_generators.append((f'translate_second_{e}', [(a, b ^ e) for a, b in lines]))
dual_generators += [
    ('scale_first', [(mul(2, a), b) for a, b in lines]),
    ('scale_second', [(a, mul(2, b)) for a, b in lines]),
    ('swap', [(b, a) for a, b in lines]),
    ('frobenius', [(mul(a, a), mul(b, b)) for a, b in lines])]
object_generators = []
for name, images in dual_generators:
    lm = [line_index[L] for L in images]
    pm = [dual_line_index[frozenset(lm[m] for m in M)] for M in dual_lines]
    perm = []
    for t, obj in objects:
        image = pm[obj] if t == 0 else lm[obj] if t == 1 else (pm[obj[0]], lm[obj[1]])
        perm.append(index[t, image])
    require(len(set(perm)) == len(objects), "nonbijective action: " + name)
    for i in range(len(objects)):
        require({perm[j] for j in neighbours[i]} == neighbours[perm[i]],
                "action does not preserve incidence: " + name)
    object_generators.append(perm)
remaining = set(chambers)
chamber_orbits = []
while remaining:
    root = min(remaining)
    orbit, queue = {root}, deque([root])
    remaining.remove(root)
    while queue:
        C = queue.popleft()
        for perm in object_generators:
            D = tuple(perm[i] for i in C)
            require(D in chambers, "nonchamber image")
            if D not in orbit:
                require(D in remaining, "overlapping orbits")
                remaining.remove(D)
```

```
                    orbit.add(D)
                    queue.append(D)
        chamber_orbits.append(len(orbit))
    require(len(chamber_orbits) == 1, "displayed subgroup is not chamber transitive")
    is_rc = all(len(pattern) == 1 for counts in residue_patterns for pattern in counts)
    require(is_rc == (q == 8), "unexpected residual-connectedness outcome")
    if q == 4:
        require(residue_patterns[1] == {(6, 6, 6):16}, "q4 line residues must be 3C6")
    encoding = {'points':points, 'lines':lines, 'source_line_sets':[sorted(M) for M in line_sets],
                'objects':objects, 'neighbours':[sorted(N) for N in neighbours],
                'object_generators':object_generators}
    return {'q':q, 'polynomial_bits':field.modulus, 'field_axioms_checked':True,
            'objects_by_type':[len(T) for T in bytype],
            'all_flag_counts':[len(F) for F in allflags],
            'all_flags_extend':True, 'firm':True,
            'panel_completion_distribution':dict(sorted(panel_counts.items())),
            'whole_incidence_graph_connected':True,
            'rank_two_residue_component_patterns':[
                [{'component_sizes':list(P),'number_of_residues':n} for P,n in sorted(C.items())]
                for C in residue_patterns],
            'residually_connected':is_rc, 'disconnected_examples':disconnected_examples,
            'verified_subgroup_generators':[name for name,_ in dual_generators],
            'chamber_orbits_under_verified_subgroup':chamber_orbits,
            'flag_transitive':True,
            'full_automorphism_group_computed':False,
            'certificate_sha256':hashlib.sha256(json.dumps(encoding,sort_keys=True).encode()).hexdigest()}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output', type=Path, required=True)
    args = parser.parse_args()
    require(not args.output.exists(), 'refusing existing output')
    report = {'status':'PASS', 'source':'L092 Construction 3 and Theorem 6',
              'source_type_order':'original point, original line, original flag',
              'cases':[check(q) for q in (4,8)],
              'uniform_parameter_classification_checked':False, 'Lean_certificate':False,
              'checker_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    with args.output.open('x') as f:
        json.dump(report, f, indent=2)
        f.write('\n')
    for case in report['cases']:
        print(json.dumps({key:case[key] for key in ('q','objects_by_type','all_flag_counts',
              'firm','residually_connected','chamber_orbits_under_verified_subgroup')}))


if __name__ == '__main__':
    main()
```

11. phase182_inversive_geometry_check.py

Role: validation. Literal S(3,4,10) all flags/residues; the full automorphism assertion uses Kantor in the written proof.

```
#!/usr/bin/env python3
"""Literal Construction 2 on the F3 sublines of P1(F9).

An independently generated finite diagnostic for L092 Lemma 5. The output
is a finite certificate, not a classification of every source parameter.
"""
from collections import Counter, deque
from itertools import combinations
import argparse
import hashlib
import json
from pathlib import Path


def require(value, message):
    if not value:
        raise RuntimeError(message)
```

```
def add(x, y):
    return (x % 3 + y % 3) % 3 + 3 * ((x // 3 + y // 3) % 3)


def mul(x, y):
    a, b, c, d = x % 3, x // 3, y % 3, y // 3
    return (a*c + 2*b*d) % 3 + 3 * ((a*d + b*c) % 3)


def inv(x):
    require(x != 0, "division by zero")
    return next(y for y in range(1, 9) if mul(x, y) == 1)


def compose(a, b):
    return tuple(a[b[i]] for i in range(10))


def generated_permutations(gens):
    seen = {tuple(range(10))}
    queue = deque(seen)
    while queue:
        old = queue.popleft()
        for g in gens:
            new = compose(g, old)
            if new not in seen:
                seen.add(new)
                queue.append(new)
    return sorted(seen)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output", type=Path, required=True)
    args = parser.parse_args()
    require(not args.output.exists(), "refusing existing output")
    # F9 = F3[a]/(a^2+1), encoded x+3y; infinity has label 9.
    translation = tuple(add(x, 1) if x < 9 else 9 for x in range(10))
    scaling = tuple(mul(x, 4) if x < 9 else 9 for x in range(10))
    inversion = tuple(9 if x == 0 else 0 if x == 9 else inv(x) for x in range(10))
    pgl = generated_permutations([translation, scaling, inversion])
    require(len(pgl) == 720, "wrong PGL action")
    seed = frozenset([0, 1, 2, 9])
    blocks = sorted({tuple(sorted(g[x] for x in seed)) for g in pgl})
    require(len(blocks) == 30, "wrong subline orbit")
    triple_counts = Counter(T for B in blocks for T in combinations(B, 3))
    require(len(triple_counts) == 120 and set(triple_counts.values()) == {1},
            "not an S(3,4,10)")
    blocksets = [frozenset(B) for B in blocks]
    objects = []
    # Global object numbers carry their type explicitly, so the copied point
    # set cannot accidentally be identified with the original one.
    for x in range(10):
        objects.append((0, x))
    for B in blocksets:
        for T in combinations(sorted(B), 2):
            T = frozenset(T)
            objects.append((1, (T, B-T)))
    for B in blocksets:
        objects.append((2, B))
    for x in range(10):
        objects.append((3, x))
    bytype = [set(i for i, (t, _) in enumerate(objects) if t == k) for k in range(4)]
    neighbours = [set() for _ in objects]

    def incident(i, j):
        ti, x = objects[i]
        tj, y = objects[j]
        if ti > tj:
            return incident(j, i)
        if ti == tj:
            return i == j
        if (ti, tj) == (0, 1):
            return x in y[0]
```

```
    if (ti, tj) == (0, 2):
        return x in y
    if (ti, tj) == (0, 3):
        return x != y
    if (ti, tj) == (1, 2):
        return x[0] <= y and not (x[1] & y)
    if (ti, tj) == (1, 3):
        return y in x[1]
    if (ti, tj) == (2, 3):
        return y not in x
    raise RuntimeError("unhandled type pair")

for i, j in combinations(range(len(objects)), 2):
    if incident(i, j):
        neighbours[i].add(j)
        neighbours[j].add(i)
allflags = [[] for _ in range(5)]

def flags(prefix, choices):
    allflags[len(prefix)].append(tuple(prefix))
    for i in sorted(choices):
        flags(prefix+[i], {j for j in choices & neighbours[i] if j > i})

flags([], set(range(len(objects))))
chambers = set(allflags[4])
extended = set()
for C in chambers:
    for size in range(5):
        extended.update(combinations(C, size))
require(all(F in extended for layer in allflags for F in layer),
        "nonextendable flag in literal construction")

def residue(F):
    used = {objects[i][0] for i in F}
    remaining = set.union(*(bytype[t] for t in range(4) if t not in used))
    for i in F:
        remaining &= neighbours[i]
    return remaining

panel_counts = Counter(len(residue(F)) for F in allflags[3])
require(min(panel_counts) >= 2, "not firm")
connected_by_corank = Counter()
for size in range(3):
    for F in allflags[size]:
        R = residue(F)
        require(bool(R), "empty rank-at-least-two residue")
        seen = {min(R)}
        queue = deque(seen)
        while queue:
            i = queue.popleft()
            new = (neighbours[i] & R) - seen
            seen.update(new)
            queue.extend(new)
        require(seen == R, "disconnected residue: " + repr(F))
        connected_by_corank[4-size] += 1

# The full PGL orbit construction is independent of the incidence tests.
# Its literal point permutations induce automorphisms of all four types.
def image_object(obj, g):
    t, x = obj
    if t in [0, 3]:
        return t, g[x]
    if t == 2:
        return t, frozenset(g[p] for p in x)
    return t, (frozenset(g[p] for p in x[0]), frozenset(g[p] for p in x[1]))

index = {obj: i for i, obj in enumerate(objects)}
frobenius = tuple(mul(mul(x, x), x) if x < 9 else 9 for x in range(10))
semilinear = generated_permutations([translation, scaling, inversion, frobenius])
require(len(semilinear) == 1440, "wrong semilinear subgroup order")
automorphism_checks = 0
for g in [translation, scaling, inversion, frobenius]:
    perm = [index[image_object(obj, g)] for obj in objects]
    require(len(set(perm)) == len(objects), "object action is not bijective")
    for i in range(len(objects)):
```

```
            require({perm[j] for j in neighbours[i]} == neighbours[perm[i]],
                    "incidence action failure")
            automorphism_checks += 1
    remaining_chambers = set(chambers)
    orbit_sizes = []
    while remaining_chambers:
        C = min(remaining_chambers)
        orbit = {tuple(sorted(index[image_object(objects[i], g)] for i in C))
                 for g in semilinear}
        require(orbit <= remaining_chambers, "nonpartitioned chamber orbit")
        orbit_sizes.append(len(orbit))
        remaining_chambers -= orbit
    encoding = {'blocks':blocks,'objects':[(t, x if t in [0,3] else
                  sorted(x) if t == 2 else [sorted(x[0]), sorted(x[1])])
                  for t, x in objects],
                'neighbours':[sorted(N) for N in neighbours]}
    report = {'status':'PASS','source':'L092, Construction 2 and Lemma 5',
              'parameters':{'q':3,'e':2,'t':3,'k':4,'v':10},
              'field':'F3[a]/(a^2+1); elements x+3y; infinity=9',
              'group_generators':[translation,scaling,inversion],
              'PGL_order':len(pgl),'semilinear_subgroup_order':len(semilinear),
              'blocks':len(blocks),'steiner_triples':len(triple_counts),
              'object_counts':[len(S) for S in bytype],
              'all_flag_counts':[len(F) for F in allflags],
              'all_flags_extend':True,'panel_completion_distribution':dict(panel_counts),
              'connected_residues_by_corank':dict(connected_by_corank),
              'incidence_automorphism_vertex_checks':automorphism_checks,
              'semilinear_chamber_orbit_sizes':sorted(orbit_sizes),
              'full_geometry_automorphism_group_computed':False,
              'full_design_automorphism_group_computed':False,
              'literal_Lemma_5_residual_connectedness_counterexample':True,
              'does_not_refute_Theorem_11_flag_transitive_classification':True,
              'Lean_certificate':False,
              'certificate_sha256':hashlib.sha256(json.dumps(encoding,sort_keys=True).encode()).hexdigest(),
              'checker_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    with args.output.open('x') as f:
        json.dump(report,f,indent=2);f.write('\n')
    print(json.dumps(report,indent=2))


if __name__ == '__main__':
    main()
```

12. hermitian_unital_check.g

Role: validation. Literal projective unital coordinates q2/3/4/5, representative residues and root orbits.

```
# Independent exact projective-coordinate construction checks.
# Run: gap -q research/leemans/hermitian_unital_check.g
LogTo("research/leemans/hermitian_unital_check.log");

ConnectedBipartite := function(left,right,inc)
 local nl,nr,adj,i,j,seen,queue,pos,v,w;
 nl:=Length(left); nr:=Length(right);
 adj:=List([1..nl+nr],x->[]);
 for i in [1..nl] do for j in [1..nr] do
  if inc(left[i],right[j]) then
   Add(adj[i],nl+j); Add(adj[nl+j],i);
  fi;
 od; od;
 seen:=List([1..nl+nr],x->false); seen[1]:=true;
 queue:=[1]; pos:=1;
 while pos<=Length(queue) do
  v:=queue[pos];pos:=pos+1;
  for w in adj[v] do if not seen[w] then
   seen[w]:=true;Add(queue,w);
  fi;od;
 od;
 return rec(connected:=ForAll(seen,x->x),vertices:=nl+nr,
            edges:=Sum(adj,Length)/2,
            degrees:=Set(List(adj,Length)));
end;
```

```
CheckUnital := function(q)
 local K,elts,z,o,pv,x,y,pts,blocks,c,B,n,Q,L,p,M,flag,
       outside,flags,R,pr,lr,fr,global,delta,ts,gens,H,X,g,v,
       shellnorm,rootmoves,orbit;
 K:=GF(q^2);elts:=Elements(K);z:=Zero(K);o:=One(K);
 pv:=[];
 for x in elts do for y in elts do Add(pv,[o,x,y]);od;od;
 for x in elts do Add(pv,[z,o,x]);od;
 Add(pv,[z,z,o]);
 pts:=Filtered(pv,v->Sum(v,x->x^(q+1))=z);
 blocks:=[];
 for c in pv do
  B:=Filtered([1..Length(pts)],i->Sum([1..3],j->pts[i][j]*c[j])=z);
  if Length(B)=q+1 then Add(blocks,B);fi;
 od;
 n:=Length(pts);
 if n<>q^3+1 or Length(blocks)<>q^2*(q^2-q+1) then Error("Design sizes");fi;
 if not ForAll([1..n],i->Number(blocks,B->i in B)=q^2) then Error("Point degrees");fi;
 for x in [1..n] do for y in [x+1..n] do
  if Number(blocks,B->x in B and y in B)<>1 then Error("Pair uniqueness");fi;
 od;od;
 Q:=1;L:=blocks[1];
 # Residue of a point: distinct flag vertices identify with all p<>Q.
 pr:=ConnectedBipartite(Difference([1..n],[Q]),Filtered(blocks,B->not Q in B),
                        function(p,B) return p in B; end);
 # Residue of a block: flags (p,M) with p in L and M<>L.
 flags:=[];
 for p in L do for M in blocks do
  if p in M and M<>L then Add(flags,[p,M]);fi;
 od;od;
 outside:=Difference([1..n],L);
 lr:=ConnectedBipartite(outside,flags,
             function(Q,f) return Q in f[2] and Q<>f[1];end);
 # Residue of an original flag (p,M).
 p:=L[1]; M:=L;
 fr:=ConnectedBipartite(Difference(M,[p]),
       Filtered(blocks,B->p in B and B<>M),
       function(Q,B) return not Q in B;end);
 global:=ConnectedBipartite([1..n],blocks,function(Q,B)return not Q in B;end);
 if not pr.connected or not lr.connected or not fr.connected or not global.connected
 then Error("Disconnected geometry/residue");fi;
 # Independent root-group orbit check, in hyperbolic Hermitian coordinates.
 delta:=First(elts,t->t<>z and t+t^q=z);
 ts:=Filtered(elts,t->t+t^q=z);
 gens:=Concatenation(List(ts,t->[[o,t],[z,o]]),List(ts,t->[[o,z],[t,o]]));
 H:=Group(gens);
 shellnorm:=v->v[1]*v[2]^q+v[1]^q*v[2];
 X:=[];
 for x in elts do for y in elts do
  if shellnorm([x,y])=-o then Add(X,[x,y]);fi;
 od;od;
 if Size(H)<>q*(q^2-1) or Length(X)<>Size(H) then Error("SU2/shell size");fi;
 orbit:=Orbit(H,X[1],OnRight);
 if Set(orbit)<>Set(X) then Error("Root group not transitive on shell");fi;
 rootmoves:=ForAll(gens,g->ForAll(X,v->shellnorm(v*g-v)=z));
 if not rootmoves then Error("Root move non-isotropic");fi;
 Print("q=",q," points=",n," blocks=",Length(blocks),"\n");
 Print("point_residue=",pr,"\nblock_residue=",lr,
       "\nflag_residue=",fr,"\nantiincidence=",global,"\n");
 Print("SU2_size=",Size(H)," norm_shell=",Length(X),
       " root_group_orbit=",Length(orbit)," isotropic_moves=",rootmoves," PASS\n");
end;

for q in [2,3,4,5] do CheckUnital(q);od;
LogTo();
QUIT;
```

13. phase199_path_length.py

Role: validation. Literal AG(2,3) distinguishes three-edge chamber action and four-edge path action.

```
#!/usr/bin/env python3
"""Literal AG(2,3) distinguishes three-edge from four-edge transitivity."""
```

```
from collections import deque
from itertools import combinations, product
import json


def require(test, message):
    if not test:
        raise ValueError(message)


def connected(vertices, neighbors):
    vertices = set(vertices)
    if not vertices:
        return False
    seen = {next(iter(vertices))}
    queue = deque(seen)
    while queue:
        x = queue.popleft()
        for y in neighbors[x] & vertices - seen:
            seen.add(y)
            queue.append(y)
    return seen == vertices


def main():
    points = list(product(range(3), repeat=2))
    index = {p:i for i,p in enumerate(points)}
    lines = sorted({tuple(sorted(index[((a+t*x)%3, (b+t*y)%3)] for t in range(3)))
                    for a,b in points for x,y in points if (x,y) != (0,0)})
    flags = [(p,l) for l,L in enumerate(lines) for p in L]
    require(len(lines) == 12 and len(flags) == 36, 'affine plane counts')
    sets = [set(L) for L in lines]
    objects = [(0,p) for p in range(9)] + [(1,l) for l in range(12)] + [(2,f) for f in range(36)]

    def incident(a,b):
        if a[0] > b[0]:
            a,b = b,a
        if a[0] == b[0]:
            return False
        if a[0:1] == (0,) and b[0:1] == (1,):
            return a[1] not in sets[b[1]]
        x,m = flags[b[1]]
        if a[0] == 0:
            return a[1] in sets[m] and a[1] != x
        return sets[a[1]] & sets[m] == {x}

    neighbors = {x:{y for y in objects if incident(x,y)} for x in objects}
    chambers = {(p,l,f) for p in range(9) for l in range(12) for f in range(36)
                if incident((0,p),(1,l)) and incident((0,p),(2,f)) and incident((1,l),(2,f))}
    panels = []
    for a,b in combinations(objects,2):
        if not incident(a,b):
            continue
        completions = neighbors[a] & neighbors[b]
        require(len(completions) >= 2, 'nonfirm or nonextending pair')
        panels.append(len(completions))
    require(all(neighbors[x] for x in objects), 'isolated object')
    require(connected(objects, neighbors), 'disconnected whole geometry')
    for x in objects:
        residue = neighbors[x]
        require(connected(residue, neighbors), 'disconnected rank-two residue')
        types = sorted({y[0] for y in residue})
        left = {y for y in residue if y[0] == types[0]}
        right = residue-left
        digon = all(right <= neighbors[y] for y in left)
        linear = all(len(neighbors[a] & neighbors[b] & right) <= 1
                     for a,b in combinations(left,2))
        require(digon or linear, 'rank-two intersection property')
    group = []
    for a,b,c,d in product(range(3),repeat=4):
        if (a*d-b*c)%3 != 1:
            continue
        for u,v in points:
            group.append(tuple(index[((a*x+b*y+u)%3,(c*x+d*y+v)%3)] for x,y in points))
    require(len(group) == len(set(group)) == 216, 'ASL order')
```

```
    line_maps = []
    flag_maps = []
    for g in group:
        lm = [lines.index(tuple(sorted(g[x] for x in L))) for L in lines]
        fm = [flags.index((g[p],lm[l])) for p,l in flags]
        line_maps.append(lm)
        flag_maps.append(fm)
    p,l,f = next(iter(chambers))
    orbit = {(g[p],lm[l],fm[f]) for g,lm,fm in zip(group,line_maps,flag_maps)}
    require(orbit == chambers and len(chambers) == 216, 'not regular chamber action')
    original_flag = flags[0]
    require(len({(g[original_flag[0]],lm[original_flag[1]]) for g,lm in zip(group,line_maps)}) == 36,
            'original flag transitivity')
    paths3 = {(p,m,x,l) for p in range(9) for m,M in enumerate(lines) for x in M
              for l,L in enumerate(lines) if p in M and p != x and x in L and m != l}
    paths4 = {path+(y,) for path in paths3 for y in lines[path[3]] if y != path[2]}
    require(len(paths3) == 216 and len(paths4) == 432, 'path counts')
    p,m,x,l,y = next(iter(paths4))
    orbit4 = {(g[p],lm[m],g[x],lm[l],g[y]) for g,lm in zip(group,line_maps)}
    require(len(orbit4) == 216 and orbit4 < paths4, 'four-edge counterexample')
    print(json.dumps(dict(status='PASS',q=3,object_counts=[9,12,36],chambers=216,
        all_flags_extend=True,firm_panel_sizes=sorted(set(panels)),connected_residues=58,
        rank_two_intersection_checks=57,ASL_order=216,ASL_regular_on_chambers=True,
        ASL_original_flag_transitive=True,three_edge_paths=216,four_edge_paths=432,
        four_edge_orbit_size=216,
        conclusion='Chamber transitivity is three-edge/four-vertex transitivity; the conventional four-edge
        interpretation of the source claim is false.'),indent=2))


if __name__ == '__main__':
    main()
```